AARHUS UNIVERSITY
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

Term:	FALL 2025
Examination:	Declarative Programming
Date:	16 December 2025
Duration:	3 hours

- This is the exam for the lecture Declarative Programming. It consists of 18 questions. You have 3 hours to answer all of them. Strategically, you should first scan the exam paper quickly in its entirety, and identify the groups of questions that seem easiest to you. ***You should answer those first*** and then move on to the others.
- For each question it is indicated how and where the answer must be provided. There are questions that require writing a calculation or the result of a calculation, and questions that require you to write Ocaml or Prolog code, in particular. Only what is written or drawn inside the specified areas is considered as your answer.
- **If you are in doubt about what is expected, please ask!**
- Each question has a weight **[w]**, where **w** is an integer number, specifying the number of points awarded for that question. You can find the weight at the top right corner of each question. The total number of points of all questions together is 100.

Question 1 **[6]**

Recall the signature of the functions (+) and (=)

```
val (+) : int -> int -> int
val (=) : 'a -> 'a -> bool
```

What are the signatures of the functions `f`, `g` and `h`?

```
let f b a = b
let g c b a = a + c
let h a c b = c = (a = b)
```

Answer:

`val f :` 'a -> 'b -> 'a

`val g :` int -> 'a -> int -> int

`val h :` 'a -> bool -> 'a -> bool

Question 2 **[4]**

(a) Define a function with signature `val invert : int -> int -> int` that, when applied to the values k, x, yields $-1 * k * x$, e.g.,

```
-50 = invert 5 10
```

Answer: `let` invert k x = - 1 * k * x

(b) Define a function with signature

```
val tr_sc : int * int -> int -> int * int -> int * int = <fun>
```

that takes a pair of integers (tx, ty), an integer k and a pair of integers (x, y) returns a new pair $(k * (x + tx), k * (y + ty))$, e.g.:

```
(3, 3)  =  tr_sc (0,0) 3 (1,1)
(5, 30)  =  tr_sc (0,5) 5 (1,1)
```

Answer: `let` tr_sc (tx, ty) k (x, y) = (k * (x + tx), k * (y + ty))

Question 3 **[6]**

Complete the following recursive function `act es stk` so that it implements the following behaviour:

```
act ["unlock";"open"] "locked" = "opened"
act ["close";"lock"] "opened" = "locked"
act ["open";"open";"open"] "locked" = "locked"
```

```
let rec act es st =
match es, st with
  [], stk                                  -> stk
  | ["lock"; _], "closed"                  -> act es' "locked"
  | ["close"; _], "open"                   -> act es' "closed"
  | ["open"; _], "closed"                  -> act es' "opened"
  | ["close"; _], "closed"                 -> act es' "closed"
  | [_; _], stk                            -> act es' st
```

Question 4 **[6]**

You are given the following recursive implementation of function `isshift v xs i` that checks whether list `xs` is shifted by `v`. E.g.,

```
isshift 1 [1] 0 = true
isshift 1 [2; 3] 1 = true
isshift 2 [2; 3] 1 = false
```

```
let rec isshift v xs i =
  match xs with
    x::xs' -> i + v = x && isshift v xs' (i + 1)
  | [] -> true
```

Complete the tail recursive function `isshift_tr v xs i ac` below that behaves just as the function `isshift v xs i` such that `isshift_tr v xs i true` = `isshift v xs i`. Hint: The argument `ac` is the accumulator.

```
let rec isshift_tr v xs i ac =
  match xs with

    x::xs' ->  isshift_tr v xs (i + 1) (ac && i + v = x)

  | [] ->      ac
```

Question 5 **[4]**

Functions `collect` and `abs` are defined as

```
let rec collect f xs =
  match xs with
    [] -> []
  | x::xs' -> if f x > 0 then x::collect f xs' else collect f xs'

let abs x = if x < 0 then -x else x
```

(a) Using functions `collect` and `abs`, implement function `filter_0 xs` that returns a list of all elements of `xs` that are not equal to `0`. Hints: You need lambda abstraction to implement this. Pay attention to the types.
Answer:

```
let filter_0 =  fun xs -> collect abs xs
```

(b) Using function `collect`, implement a function `filter_v v xs` that returns a list of all elements of `xs` that are larger than `v`
Answer:

```
let filter_v v =  fun xs -> collect (fun x -> x - v) xs
```

Question 6 **[7]**

(a) Unnest the following function `cargo`

```
let cargo c =
  let cult w a =
    let science s w =
      c + s - w in
    science w a - c in
  cult 1 c
```

Answer:

```
let science s w c = c + s - w
let cult w a c = science w a c - c
let cargo c = cult 1 c c
```

(b) Unnest the following function `this`

```
let this f =
  let is x f =
    let that g =
      g x * f x in
    that (fun x -> f x - 1) in
  is (f 0) f
```

Answer:

```
let that g x f = g x * f x
let is x f = that (fun x -> f x - 1) x f
let this f = is (f 0) f
```

Hint: Function declarations can be nested, that is, “inner” functions are declared locally to “outer” functions. Parameters to outer functions can be referred to in inner functions.
Nested functions can be unnested by declaring all “inner” functions globally before the outer function making the passed parameters explicit. The behaviours of the nested and corresponding unnested functions are indistinguishable.

Question 7 **[5]**

Given the type `func` below

```
type func = Z | N of func | S of func * func
```

and the recursive functions `calc` and `div`

```
let rec calc e =
  match e with
    S (Z, x) -> x
  | S (x, Z) -> calc (S (Z, x))
  | S (x, N y) -> calc (S (N x, y))
  | _ -> e

let rec div e =
  match e with
    N (N x) -> N (div x)
  | _ -> e
```

We can compute

```
div (calc (S (N (N Z), N (N Z)))) = N (N Z)
```

Reading the function definitions as a system of equations as we have done in the course, calculate the result of the computation by inserting the missing steps:

```
div (calc (S (N (N Z), N (N Z))))
```

= div (calc (S (N (N (N Z)), N Z)))

= div (calc (S (N (N (N (N Z))), Z)))

= div (calc (S (Z, N (N (N (N Z)))))))

= div (N (N (N (N Z))))

= N (div (N (N Z)))

= N (N (div Z))

= `N (N Z)`

Question 8 **[5]**

(a) Given the definitions of the functions `flipbit`, `map`, `sum` and `bitop`,

```
let flipbit x = (* Inverts a bit *)
  match x with
    true  -> false
  | false -> true

let rec map f xs =
  match xs with
  | x::xs' -> f x::map f xs'
  | _ -> []

let rec sum xs ys carry = (* Adds binary numbers xs and ys *)
  match (xs, ys) with      (* with carry *)
    (true::xs', false::ys')
  | (false::xs', true::ys') -> not carry::sum xs' ys' carry
  | (false::xs', false::ys') -> carry::sum xs' ys' false
  | (true::xs', true::ys') -> carry::sum xs' ys' true
  | _ -> []

let bitop op xs ys = op xs ys false
```

implement a function `sumone xs` using functions `bitop`, `sum` and `map` that adds binary number `[true; ...; true]` (of the same length as `xs`) to `xs`

```
[false; false] = sumone [true; false]
[true; true; true] = sumone [false; false; false]
[false; true; true] = sumone [true; true; true]
```

Answer:

`let sumone =` fun xs -> bitop sum xs (map (fun x -> true) xs)

Note: You are not allowed to add parameters before the `=`.
Hint: You need lambda abstraction to implement this.

(b) Use functions `bitop`, `sum`, `map` and `flipbit` to implement a function `sumflip xs` that adds a binary number, where each bit of `xs` is flipped, to `xs` such that

```
[true; true] = sumflip [true; false]
[true; true; true] = sumflip [false; false; false]
[true; true; true] = sumflip [true; true; true]
[true; true; true; true] = sumflip [true; false; true; false]
```

Answer:

`let sumflip =` fun xs -> bitop sum xs (map flipbit xs)

Note: You are not allowed to add parameters before the `=`.

Question 9 **[8]**

For a list, e.g.,

```
[0; 0; 1; 0; 0; 1]
```

the following function `ckone` checks whether the list has an even number of elements and the second argument is a `1`; function `cknul` checks whether the list has an uneven number of elements and the second argument is a `0`.

```
let rec ckone xs x =
  match xs with
    [] -> x = 1
  | x::xs' -> cknul xs' (1 - x)
and cknul xs x =
  match xs with
    [] -> x = 0
  | x::xs' -> ckone xs' (1 - x)
```

Considering the function definitions as a system of equations, calculate:

(a) The value of `ckone [0; 0; 1] 0`

`ckone [0; 0; 1] 0`

= cknul [0; 1] 1

= ckone [1] 1

= cknul [] 0

= x = 0

(b) The value of `cknul [1; 0; 1] 0`

`cknul [1; 0; 1] 0`

= ckone [0; 1] 0

= cknul [1] 1

= ckone [] 0

= x = 1

Question 10 **[6]**

Consider the pattern matching algorithm match from the course

$$
\begin{array}{lll}
\mathrm{match}\ S\ u\ =\ u & & \text{if } S = \emptyset \\
\qquad\qquad \mathrm{match}\ (S \setminus (x,t))\ u & & \text{if } (x,t) \in S \cap u \\
\qquad\qquad \mathrm{match}\ (S \setminus (x,t))\ (u \cup \{(x,t)\}) & & \text{if } (x,t) \in S \wedge (x,s) \notin u \text{ for all } s \\
\qquad\qquad \mathrm{match}\ (S \cup \{(s_1,s'_1),\ldots,(s_k,s'_k)\}\ u & & \text{if } (c(s_1,...,s_k), c'(s'_1,...,s'_k)) \in S \wedge c = c' \\
\qquad\qquad \mathbf{fail} & & \text{otherwise}
\end{array}
$$

Let small letters a, b, c, ... denote variables, and capital letters A, B, C, ... constants. As usual, we leave away empty argument lists, e.g., instead of $Z()$ we write Z.

Calculate:

(a) The value of $\mathrm{match}\ \{(U(w, X(v,u), X(T,v)),\ U(Y, X(c, X(a)), X(T,c)))\}\ \emptyset$

$\mathrm{match}\ \{(U(w, X(v,u), X(T,v)),\ U(Y, X(c, X(a)), X(T,c)))\}\ \emptyset$

= match {(w,Y), (X(v,u),X(c,X(a))), (X(T,v),X(T,c))} Ø

= match {(w,Y), (v,c), (u,X(a)), (X(T,v),X(T,c))} Ø

= match {(w,Y), (v,c), (u,X(a)), (T,T), (v,c)} Ø

= match {(v,c), (u,X(a)), (T,T), (v,c)} {(w,Y)}

= match {(u,X(a)), (T,T), (v,c)} {(w,Y), (v,c)}

= match {(T,T), (v,c)} {(w,Y), (v,c), (u,X(a))}

= fail

=

=

(b) The value of $\mathrm{match}\ \{(U(U(a,c),a),\ U(U(T,Z),U(T)))\}\ \emptyset$

$\mathrm{match}\ \{(U(U(a,c),a),\ U(U(T,Z),U(T)))\}\ \emptyset$

= match {(U(a,c),U(T,Z)), (a,U(T))} Ø

= match {(a,T), (c,Z), (a,U(T))} Ø

= match {(c,Z), (a,U(T))} {(a,T)}

= match {(a,U(T))} {(a,T), (c,Z)}

= fail

Question 11 **[5]**

Using the truth table method, determine whether the formula

$$(\neg(P \Rightarrow Q)) \Rightarrow Q$$

is satisfiable.

Fill in the truth table:

P	Q	$(\neg(P \Rightarrow Q)) \Rightarrow Q$
T	T	T
T	F	F
F	T	T
F	F	T

Tick the correct answer:

☒ The formula is satisfiable.

☐ The formula is unsatisfiable.

Question 12 **[4]**

Determine whether the formula

$$\begin{array}{ll} & Q \vee \neg H \vee \neg P \vee R \\ \wedge & \neg Q \vee R \\ \wedge & P \vee Q \vee R \\ \wedge & Q \vee \neg R \end{array}$$

is satisfiable applying the DPLL algorithm, using the rules

(UR) the 1-literal rule (aka unit clause rule),
(PR) the affirmative-negative rule (aka pure literal rule),
(SR) the splitting rule.

Fill into the boxes the formulas resulting from the rule applications indicated between the boxes.

1:

$$\begin{array}{ll} & Q \vee \neg H \vee \neg P \vee R \\ \wedge & \neg Q \vee R \\ \wedge & P \vee Q \vee R \\ \wedge & Q \vee \neg R \end{array}$$

Applying rule **(PR)** for $\neg H$ to the formula in box 1 yields:

2:

¬Q ∨ R
∧ P ∨ Q ∨ R
∧ Q ∨ ¬R

Applying rule **(SR)** for Q to the formula in box 2 yields:

3:

R

Applying rule **(UR)** to the formula in box 3 yields:

4:

T

Tick the correct answer:
☒ The formula is satisfiable.
☐ The formula is unsatisfiable.

Question 13 **[6]**

The following table specifies two lambda terms in the first row. Complete the empty cells in the remaining rows of the table according to the lambda terms for each column.
Remark: In your answers you may write “L” to represent “λ” if you prefer.

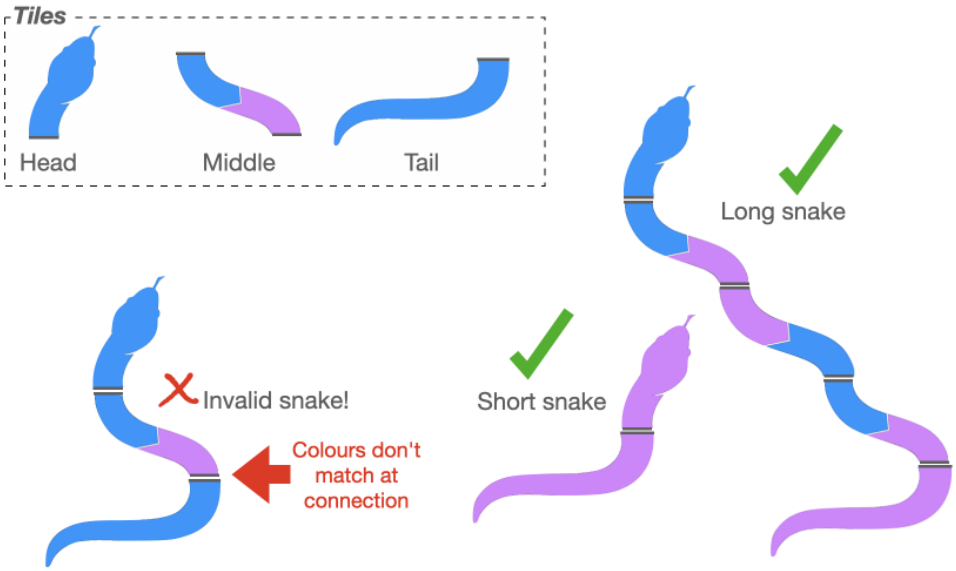
Lambda term	$\lambda f \ \cdot ((\lambda x \cdot f\ (x\ x))(\lambda x \cdot f\ (x\ x)))$	$\lambda x \ \cdot (f\ (\lambda vw \cdot x\ x\ w))$
Name the **type** of lambda term	abstraction	abstraction
Rewrite the term in its fully curried form	L f · ((Lx · f (x x))(Lx · f (x x)))	Lx · (f (Lv · (Lw · x x w)))
Specify the **free** variables	none	f
Specify the **bound** variables	f, x	x, v, w

Question 14 **[8]**

Snake tiles is a game where the player gets a random set of "snake tiles" and needs to assemble them to create the longest snake possible. Tiles can be:

- a *head* tile, coloured either *pink* or *blue*
- a *tail* tile, coloured either *pink* or *blue*
- a *middle section* tile that always has one part *pink* and the other part *blue*

Tiles can only be placed next to each other if their colours match on the sides that the tiles touch each other. The illustration below shows the tile shapes and gives some examples of valid and invalid snakes:



This game is modelled in the following Prolog program. A snake is modelled as a list of parts ($head/1$, $middle/2$, $tail/1$). The predicate $snake/1$ is True when the argument is a valid snake, and $snakeBody/1$ is True when the argument is a valid snake without a head:

```
snake( [head(C) | Body] ) :-
  snakeBody(Body),
  colour(Body, C).

snakeBody( [tail(_)] ).
snakeBody( [middle(_,C) | Body] ) :-
  snakeBody(Body),
  colour(Body, C).

colour( [tail(C)], C).
colour( [middle(C,_)|_], C).
```

(a) For each of the following Prolog queries applied to the above program, write down the first query result when executed in Prolog. For example:

```
?- snakeBody( [tail(pink)] ).
```

First query result when executed in Prolog:

```
true.
```

(i) `?- colour( [middle(pink,blue), tail(blue)], C).`

First query result when executed in Prolog:

C = pink.

(ii) `?- snake( [head(pink), tail(pink)]).`

First query result when executed in Prolog:

true.

(iii) `?- S = [D, middle(pink,C), middle(blue, pink), A], snake(S).`

First query result when executed in Prolog:

D = head(pink), C = blue, A = tail(pink).

(b) Now we want to define a predicate that will assemble valid snakes from a set of given tiles. Firstly you need to define a predicate remove(X,List,NewList) that is True when `X` is a member of the list `List` and the list `NewList` is identical to `List` except one instance of `X` has been removed. For example:

`?- remove(X, [a,b,c], [a,c]).`

...is True when `X = b`.

Hint: Consider the following definition of $remove/3$. The original list `List` is a *concatenation* of some list A with another list that has a head X and tail B. If X is removed from `List`, then the new list `NewList` is A concatenated with B.

```
remove(X, List, NewList) :- _____
```

member(X, List), append(A, [X | B], List), append(A, B, NewList).

(c) You will now implement predicates needed to define $build/2$. The predicate `build(P,S)` is True when S is a valid snake (i.e. S satisfies the predicate $snake/1$) and is assembled from tiles given in the list P. For example:

`?- P = [tail(pink), head(pink),middle(pink, blue),tail(blue)], build(P,S).`

...delivers a snake `S = [head(pink), tail(pink)]`, and more valid snakes can be delivered on backtracking.

Consider the following incomplete program that only partly implements the $build/2$ predicate.

```
build(Parts, Snake) :-
  build_(Parts, [], Snake).
```

Using the predicates $remove/3$, $snake/1$ and $snakeBody/1$, implement the predicate $build_/3$ consisting of a base case and a recursive case. The predicate should incrementally assemble a valid snake body (in the recursive case) until it finally puts on the head and delivers the assembled valid snake (in the base case).

```
build_(Parts, CurSnake, FinalSnake ) :-
Parts = [head(C)], remove(X, Parts, _) append([X], CurSnake, FinalSnake),
snake(FinalSnake).

build_(Parts, CurSnake, FinalSnake ) :-
X = tail(_), remove(X, Parts, NewParts), append([X], CurSnake, FinalSnake),
snakeBody(FinalSnake), build_(NewParts, FinalSnake, FinalSnake);
X = middle(_, _), remove(X, Parts, NewParts), append([X], CurSnake, FinalSnake),
snakeBody(FinalSnake), build_(NewParts, FinalSnake, FinalSnake).
```

(d) Complete the following query that will find the longest possible valid snake that can be assembled using the given list of parts. *Hint:* recall the built-in predicate `length(List,Len)` is True when integer Len is the number of elements in the list $List$.

```
?- Parts = [head(pink), middle(pink, blue),
            middle(blue, pink), tail(blue), tail(pink)],
build(Parts, Snake), length(Snake, 5).
```

Question 15 **[4]**

(a) Write down the first result of each of the following Prolog queries that use built-in list predicates.

```
?- append(A, [s], [x,s]).
```

First query result when executed in Prolog:

A = [x].

```
?- K = [k(tina,5), k(sam,3), k(albert,6), k(jo,4)],
   member(k(_,A), K), member(k(_,B),K), A < B.
```

First query result when executed in Prolog:

A = 5, B = 6.

(b) Write down the result of executing the following unification queries in Prolog.

```
?- circle(point(X,3), radius(R)) =
   circle(point(Y,Y), radius(5)).
```

First query result when executed in Prolog:

X = 3, Y = 3, R = 5.

```
?- [ [gold, [jewels, gems] ], [coins, crown] ] = [H|T].
```

First query result when executed in Prolog:

H = [gold, [jewels, gems]], T = [[coins, crown]].

Question 16 **[6]**

(a) As discussed in the lectures, symbolic natural numbers can be encoded in Prolog, with 0 being encoded as the atom `z`, natural number 1 being encoded as the compound term `s(z)` and, in general, natural number n (for $n > 0$) being encoded as the compound term `s(N0)` where `N0` is the encoding of the natural number $n - 1$.

You will now define predicates that implement the *modulo operator* (mod) on symbolic natural numbers. An example of modular arithmetic is the hour hand on a clock: the hour always stays within the range 0 to 11, and when the hour hand reaches the number 12 (called the *modulus*) it "wraps around" back to 0. Some examples of the modulo operator applied to *symbolic* naturals are.:

- $s(z)$ mod $s(s(z)) = s(z)$
- $s(s(z))$ mod $s(s(z)) = z$
- $s(s(s(z)))$ mod $s(s(z)) = s(z)$

The modulo operation can be defined recursively as follows:

- a mod $b = a$, if $a < b$ and $b > 0$
- a mod $b = (a - b)$ mod b, if $a \geq b$ and $b > 0$

Use this definition to implement the predicate $mod/3$ such that **`mod`**`(A,B,C)` is True when A mod $B = C$, for symbolic natural numbers A, B, C.

Hint: You will also need to recursively define predicate $lt/2$ that implements the *less than* ($<$) comparator between two symbolic naturals, and recursive predicate $sub/3$ that implements subtraction of one symbolic natural from another, i.e. `sub(A,B,C)` is True when $A - B = C$. You will need to implement Prolog facts or rules for the base cases, and Prolog rules for the recursive cases. Recall that in Prolog `not(P)` is True when predicate `P` is False.

Answer:

```
sub(A,z,A).

sub(A,S(B'),C) :- sub(A,B',S(C)).

lt(z,B) :- not(B=z).

lt(S(A'),S(B')) :- lt(A',B').

mod(A,B,A) :- lt(A, B), lt(z, B).

mod(A, B, C) :- sub(A,B,Diff), mod(Diff, b, C), (lt(b, a); a = b), lt(z, b).
```

Question 17 **[4]**

In the lectures we discussed the unification procedure, consisting of six rewriting "rules" that step-by-step transform a set of unification expressions until arriving at a "solved substitution" or a failure.

The six rules are: *swap*, *eliminate*, *decompose*, *conflict*, *occurs*, *delete*.

Apply the same procedure to unify the expressions in (a) and (b) below. Apply exactly one "rule" at each step, and specify the rule used by writing the rule name to the right side of the new unification expression set (as we did in lectures).
Remark: In your answers you can write "$=$" to represent "$\approx$" if your prefer.

(a) Unify the term `p(W,w(w(p),x(W)))` with the term `p(x(w),w(X,x(x(Y))))`.

Answer:

$\{p(W, w(w(p), x(W))) \approx p(x(w), w(X, x(x(Y))))\}$

→ {W ≈ x(w), w(w(p),x(W)) ≈ w(X,x(x(Y)))} decompose

{W ≈ x(w), w(p) ≈ X, x(W) ≈ x(x(Y))} decompose

{W ≈ x(w), w(p) ≈ X, W ≈ x(Y)} decompose

{W ≈ x(w), w(p) ≈ X, x(w) ≈ x(Y)} eliminate W

{W ≈ x(w), w(p) ≈ X, w ≈ Y} decompose

{W ≈ x(Y), Y(p) ≈ X, w ≈ Y} eliminate w

{W ≈ x(Y), X ≈ Y(p), w ≈ Y} swap

{W ≈ x(Y), X ≈ Y(p), w ≈ Y}

(b) Unify the term `q(M,Y,Q,w,M)` with the term `q(m(Y),n(Q),w(W,C),W,C)`.

Answer:

$\{q(M, Y, Q, w, M) \approx q(m(Y), n(Q), w(W, C), W, C)\}$

→ {M ≈ m(Y), Y ≈ n(Q), Q ≈ w(W,C), w ≈ W, M ≈ C} decompose

{M ≈ m(Y), Y ≈ n(Q), Q ≈ w(W,C), w ≈ W, C ≈ M} swap

{M ≈ m(Y), Y ≈ n(Q), Q ≈ w(W,M), w ≈ W, C ≈ M} eliminate C

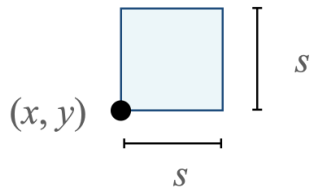
{M ≈ m(Y), Y ≈ n(w(W,M)), Q ≈ w(W,M), w ≈ W, C ≈ M} eliminate Q

{M ≈ m(n(w(W,M))), Y ≈ n(w(W,M)), Q ≈ w(W,M), w ≈ W, C ≈ M} eliminate Y

fail occurs

Question 18 **[6]**

We are creating a simple 2D box layering application for a drawing program, where boxes can *contain* other boxes. Each square box is represented by its bottom-left point (x, y), and a side length s as illustrated in the following diagram:



Consider the following Prolog program that uses the CLP(FD) library for arithmetic operators:

```
boxes( [] ).
boxes( [box(X,Y,Size)|R] ) :-
  X in 1..100, Y in 1..100, Size in 1..50,
  boxes(R).

sides( box(X,Y,Size), side(X,Size), side(Y, Size) ).

relation(contains, A, B) :-
  sides(A,AX,AY),
  sides(B,BX,BY),
  relation(contains, AX, BX),
  relation(contains, AY, BY),
  relation(bigger, A, B).

relation(bigger, box(_,_,ASize), box(_,_,BSize)) :-
  ASize #> BSize.

relation(contains, side(APos,ASize), side(BPos,BSize)) :-
  APos #=< BPos, APos + ASize #>= BPos + BSize.
```

(a) For each of the following Prolog queries applied to the above program, write down the first query result when executed in Prolog, or write down the exception that is thrown.

(i) `?- B1 = box(5, 7, 10), B2 = box(6, 5, 5), relation(R,B1,B2).`

First query result or exception when executed in Prolog:

R = bigger.

(ii)
```
?- boxes([B1,B2,B3]), relation(contains,B2,B3),
   relation(contains,B1,B2),relation(bigger,B3,B1).
```

First query result or exception when executed in Prolog:

false.

(iii) Write a query to prove that the *contains* relation is asymmetric, i.e. if box A contains box B, then it is impossible that box B also contains box A.

?- boxes([A, B]), relation(contains, A, B), relation(contains, B, A).